Guia Definitivo para a Prova

Desenvolvimento de Aplicativos Móveis (React Native + Expo)

Professor: MSc. Alex Junior Nunes **Instituição:** Universidade Positivo

Conteúdo: 1º Bimestre (Aulas 01 a 16)

1. Arquitetura e Ecossistema Mobile

1.1 Classificação dos Aplicativos

- **Nativos:** Desenvolvidos especificamente para o sistema operacional oficial.
  - **iOS:** Linguagens **Swift** e **Objective-C** na IDE **Xcode**.
  - **Android:** Linguagens **Kotlin** (oficial desde 2017) e **Java** na IDE **Android Studio**.
  - *Prós:* Alto desempenho e acesso direto aos recursos de hardware. *Contras:* Duas bases de código separadas.
- **Híbridos:** Tecnologias web (HTML, CSS, JS) envelopadas em um contêiner nativo / WebView (ex: **Ionic**, **Cordova / PhoneGap**).
- **Web Apps:** Acessados via navegador móvel, responsivos, não instalados na loja.
- **Cross-Platform (Multiplataforma):** Única base de código compartilhada entre Android e iOS:
  - **React Native (Meta/Facebook):** JavaScript/TypeScript + React. Mapeia JSX para views nativas do sistema operacional.
  - **Flutter (Google):** Linguagem **Dart**; desenha seus próprios pixels na tela usando engine gráfica (Skia/Impeller).
  - **Xamarin (Microsoft):** Linguagem C# com ecossistema .NET.

> **Casos de Mercado (Slide Context):** O **Airbnb** usou React Native por anos e migrou de volta para 100% nativo devido a necessidades extremas de performance e controle de UI de baixo nível. O **Instagram** utiliza React Native em várias telas integradas ao app nativo.

1.2 Funcionamento Interno do React Native

- **React Core:** Modelo de componentes, Virtual DOM e reconciliação.
- **React DOM:** Traduz para elementos Web (`<div>` , `<span>`).
- **React Native:** Traduz componentes JSX para views nativas do SO:
  - `<View>` → `android.view` (Android) | `UIView` (iOS) | `<div>` (Web).
  - `<Text>` → `TextView` (Android) | `UITextView` (iOS) | `<span>/<p>` (Web).
- **JS Engine:** Código JavaScript é executado fora do browser sobre motores como **V8** (Google), **WebKit** (Apple) ou **Hermes** (engine otimizada da Meta).

1.3 Expo: Managed Workflow vs. Bare Workflow

- **Managed Workflow (Gerenciado):** O desenvolvedor NÃO mexe nas pastas `/android` e `/ios` . O Expo compila e gerencia tudo de forma transparente.
- **Bare Workflow:** O projeto dá acesso total às pastas nativas `/android` e `/ios` , permitindo integração direta de código Java, Kotlin ou Swift.
- **Comandos Essenciais:** `npx create-expo-app nome --template` (selecionando minimal com bare) e `npx expo start` (com Fast Refresh).

2. Módulos ECMAScript, Fragmentos e Pastas

Tipo de Export	Sintaxe de Declaração	Sintaxe de Importação	Regras
Default	`export default function App() {}`	`import App from './App'`	Apenas 1 por arquivo; importa **sem chaves** e pode renomear.
Nomeado	`export function Botao() {}`	`import { Botao } from './Botao'`	Vários por arquivo; exige **chaves** `{}` com o nome exato.

> **Fragmentos (`<React.Fragment>` ou `<>...</>`):** Servem para agrupar múltiplos componentes filhos adjacentes sem criar um elemento visual a mais (como uma `<View>` extra) na árvore de layout nativa.

3. Flexbox no React Native (Regras e Pegadinhas)

> **REGRA CRUCIAL:** No React Native, o `flexDirection` padrão é **`column`** (vertical), diferente do CSS web tradicional (onde o padrão é `row`).

Propriedade	Eixo de Atuação	Valores	Comportamento / Pegadinha de Prova
`flexDirection`	Define os eixos	`'column'` (padrão), `'row'`	Determina quem é o Eixo Principal (Main Axis) e o Eixo Transversal (Cross Axis).
`justifyContent`	**Eixo Principal**	`'flex-start'`, `'center'`, `'flex-end'`, `'space-between'`, `'space-around'`, `'space-evenly'`	Se `flexDirection: 'column'`, alinha na vertical. Se `'row'`, alinha na horizontal.
`alignItems`	**Eixo Transversal**	`'flex-start'`, `'center'`, `'flex-end'`, `'stretch'`	**Atenção:** `'stretch'` só estica o item se ele **NÃO** tiver tamanho transversal fixo (width ou height).
`flexWrap`	Quebra de linha	`'nowrap'` (padrão), `'wrap'`	`'nowrap'` espreme os itens; `'wrap'` quebra automaticamente para a próxima linha/coluna.
`flex: 1`	Proporção	Número	Atalho para `flex-grow: 1, flex-shrink: 1, flex-basis: 0%`. Divide o espaço restante proporcionalmente.
`flexGrow`	Expansão	Número	Decide quem cresce quando **SOBRA** espaço no contêiner.
`flexShrink`	Compressão	Número	Decide quem encolhe quando **FALTA** espaço. **`flexShrink: 0`** se recusa a encolher.
`flexBasis`	Tamanho Base	px ou %	Tamanho inicial do elemento antes de distribuir o espaço livre.

4. Imagens, Ícones e Plataforma (OS)

4.1 Imagens Locais vs. Remotas (`<Image>`)

```
// 1. Imagem Local (estática via Metro)
<Image source={require('./assets/logo.png')} style={{ width: 100, height: 100 }} />

// 2. Imagem Remota (URL de rede)
<Image source={{ uri: 'https://exemplo.com/foto.jpg' }} style={{ width: 100, height: 100 }} />
```

> **PEGADINHA DE PROVA:** Em imagens remotas (`uri`), **é obrigatório informar largura (width) e altura (height)** no estilo. Se você não informar, o React Native não consegue adivinhar as dimensões da rede e a imagem fica com tamanho 0x0 (não aparece na tela!).

- `resizeMode` : `'cover'` (preenche cortando), `'contain'` (mostra tudo sem cortar), `'stretch'` (estica), `'center'` .
- **Avatar Circular:** Para uma imagem de 100×100 virar um círculo perfeito, use `borderRadius: 50` (exatamente a metade do tamanho).
- **Ícones (`@expo/vector-icons`):** `<Feather name="user" size={20} color="#fff" />` .

4.2 Diferenciação de SO com `Platform`

```
import { Platform, StyleSheet } from 'react-native';

const estilos = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    ...Platform.select({
      ios: {
        shadowColor: '#000',
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.2,
        shadowRadius: 4,
      },
      android: {
        elevation: 6, // Android usa EXCLUSIVAMENTE elevation para sombras
      },
    }),
  }
});
```

5. Listas e Performance: ScrollView vs. FlatList

Critério	`<ScrollView>`	`<FlatList>`
Renderização	Desenha **TODOS** os itens de uma vez na memória, visíveis ou não.	Usa **Virtualização**: só renderiza o que está no viewport da tela (e recicla componentes).
Uso Indicado	Telas curtas, estáticas ou formulários (~10 a 20 itens).	Listas longas, dinâmicas ou infinitas (centenas ou milhares de itens).
Histórico	Nativo do core.	Substituiu o antigo `ListView`, que foi **removido** do React Native em 2017.

As 3 Props Essenciais da `FlatList`:

```
<FlatList
  data={CONTATOS}                                  // 1. Array com os dados em memória
  keyExtractor={(item) => item.id.toString()}      // 2. Devolve string única por item
  renderItem={({ item }) => <ItemContato item={item} />} // 3. Devolve o JSX de cada linha
/>
```

Props Estruturais de UI da `FlatList`:

- `ListHeaderComponent`: Componente fixo no topo da lista.
- `ListFooterComponent`: Componente fixo no final da lista.
- `ItemSeparatorComponent`: Elemento desenhado **apenas ENTRE** os itens (nunca antes do 1º e nem depois do último).
- `ListEmptyComponent`: Renderizado automaticamente quando `data = []`.
- `horizontal={true}`: Cria lista com rolagem horizontal.

6. Comunicação entre Componentes e Destructuring

> **REGRA FUNDAMENTAL:** *"Dados descem, Eventos sobem"*.

- **Direta (Pai → Filho):** Passagem via `props`. Via de mão única: o filho recebe os dados, apenas lê e **nunca altera diretamente** a prop recebida.
- **Direta (Filho → Pai):** O pai passa uma **função de callback** como prop para o filho (ex: `aoPressionar={salvar}`). O filho executa a função ao disparar um evento (ex: clique). O filho apenas avisa que o evento ocorreu.
- **Indireta & Prop Drilling:** Quando o dado está no componente Avô e precisa chegar no Neto/Bisneto, componentes intermediários são forçados a repassar props que eles não utilizam. Isso gera código frágil e motivou a criação do **Redux** e da **Context API**.

Destructuring de Objetos (ES6)

```
const aluno = { nome: 'Higor', curso: 'ADS', endereco: { cidade: 'Curitiba' } };

// 1. Extração simples
const { nome, curso } = aluno;

// 2. Renomeação de variável
const { nome: nomeAluno } = aluno; // cria a variável 'nomeAluno' com o valor 'Higor'

// 3. Valor padrão (fallback caso a chave não exista)
const { apelido = 'Sem apelido' } = aluno;

// 4. Aninhado
const { endereco: { cidade } } = aluno; // extrai a cidade diretamente
```

7. Hooks: useState, TextInput e useEffect

7.1 Hook `useState`

Variáveis comuns (`let contador = 0`) mudam seu valor no JavaScript, mas **não avisam o React**. Para que uma alteração reflita na tela, é obrigatório usar estado reativo.

```
const [contador, setContador] = useState(0);
```

- Retorna um array com exatamente 2 elementos:
  - `[0]` (ex: `contador`): O valor atual do estado (**somente leitura**).
  - `[1]` (ex: `setContador`): A função atualizadora que agenda o re-render do componente.

> **REGRA DE OURO:** Nunca altere o estado diretamente! `contador++` ou `contador = contador + 1` é **completamente errado**. Sempre use a função set: `setContador(contador + 1)` .

7.2 Inputs: Não Controlado vs. Controlado (`<TextInput>`)

```
const [email, setEmail] = useState('');

<TextInput
  value={email}                       // Torna o input controlado pelo estado
  onChangeText={setEmail}             // Recebe o texto digitado diretamente (sem event.target)
  placeholder="Digite seu email"      // Texto informativo enquanto vazio
  keyboardType="email-address"        // Tipo de teclado ('numeric', 'email-address')
  secureTextEntry={false}             // Se true, esconde os caracteres com bolinhas (senhas)
  multiline={false}                   // Permite múltiplas linhas
  autoCapitalize="none"               // Controle de maiúsculas ('words', 'none')
/>
```

7.3 Hook `useEffect` (Efeitos Colaterais)

Um **efeito colateral (side effect)** é qualquer código que interage com o mundo fora do cálculo puro do componente: chamadas de API (`fetch`), temporizadores (`setInterval`), ouvintes de eventos e leitura/escrita em armazenamento local.

Sintaxe	Comportamento do Ciclo de Vida	Cenário de Uso
`useEffect(fn)` *(sem array)*	Executa após **toda e qualquer** renderização.	Raríssimo. Risco crítico de **loop infinito** se alterar estado dentro dele.
`useEffect(fn, [])` *(array vazio)*	Executa **uma única vez**, logo após o componente ser montado.	**Padrão ouro** para buscar dados em API (`fetch`) ao abrir a tela.
`useEffect(fn, [valor])` *(com dependências)*	Executa na montagem e **sempre que** `valor` **mudar**.	Recalcular ou recarregar dados quando um filtro, id ou estado específico for alterado.

Função de Limpeza (Cleanup): Quando o `useEffect` retorna uma função (`return () => clearInterval(id)`), ela é executada antes do efeito rodar novamente e no momento em que a tela é **desmontada**. Serve para desligar timers e evitar vazamento de memória (*memory leaks*).

8. Navegação com React Navigation

O `<NavigationContainer>` da biblioteca `@react-navigation/native` é obrigatório e deve envolver toda a estrutura de rotas na raiz (`App.js`).

Navegador	Biblioteca	Conceito / Metáfora	Quando Usar
Stack	`@react-navigation/native-stack`	Pilha de cartas (LIFO - Último a entrar é o primeiro a sair).	Fluxos sequenciais: Lista → Detalhes → Formulário de Edição.
Bottom Tabs	`@react-navigation/bottom-tabs`	Abas fixas na parte inferior da tela.	Seções principais que o usuário alterna frequentemente (Home, Busca, Perfil).
Drawer	`@react-navigation/drawer`	Menu lateral deslizante (gaveta).	Opções secundárias ou administrativas (Configurações, Termos, Sair).

> **Dependências do Drawer:** Para animação suave e gestos de arrasto, o Drawer exige obrigatoriamente: `react-native-gesture-handler` e `react-native-reanimated` .

Ações das Props `navigation` e `route` :

- `navigation.navigate('Tela', { id: 10 })` : Empilha/abre a tela informada e passa dados adicionais.
- `navigation.goBack()` : Desempilha a tela do topo e exibe a anterior com o estado preservado.
- `navigation.popToTop()` : Volta diretamente para a primeira tela da pilha.
- `navigation.openDrawer()` / `closeDrawer()` : Abre ou fecha a gaveta lateral por código.
- `navigation.getParent('NomeDoStack')?.navigate('Tela')` : Acessa o navegador pai em navegações aninhadas.
- **Recebendo parâmetros na tela de destino:**

```
function TelaDetalhes({ route }) {
  const { id, nome } = route.params || {}; // Destructuring direto dos parâmetros recebidos
  return <Text>{nome}</Text>;
}
```

- **Customização:** `screenOptions` (no Navigator) define o padrão para todas as telas; `options` (no Screen) customiza uma tela específica e tem **prioridade**.

9. Recursos Multimídia: Vídeo com `expo-video`

- **Padrão Oficial Atual:** O antigo componente `Video` do `expo-av` foi **descontinuado**. O padrão moderno do ecossistema Expo é o `expo-video`.
- **Duas Peças Fundamentais:**
  1. `useVideoPlayer(fonte, callbackConfig)`: O **cérebro**. Controla reprodução (`play()` , `pause()`), repetição (`loop = true`), som (`muted = true`) e status (`playing`).
  2. `<VideoView player={player} />`: A **tela**. Componente visual que exibe o vídeo e suporta estilos, bordas, tela cheia e PiP.
- **Vídeo Local vs. Remoto:**
  - Local: `const fonte = require('../assets/video.mp4')` (empacotado pelo Metro Bundler).
  - Remoto: `const fonte = 'https://site.com/video.mp4'` (string URL comum).
- **Props do `<VideoView>`:** `contentFit="contain"` (ou `cover`), `nativeControls={false}` , `allowsFullscreen` e `allowsPictureInPicture` .

10. Simulado de Prova com Gabarito Comentado

Questão 1: No Flexbox do React Native, qual é o valor padrão da propriedade `flexDirection` e como o `alignItems: 'stretch'` se comporta?

Gabarito Comentado: O valor padrão é `'column'` (disposição vertical). O valor `'stretch'` no `alignItems` estica os elementos ao longo do eixo transversal para preencher o contêiner, mas só terá efeito se os itens **não tiverem dimensão transversal fixa** (largura no caso de coluna, ou altura no caso de linha).

Questão 2: Por que a renderização de uma imagem remota com `<Image source={{ uri: '...' }} />` exige que largura e altura sejam declaradas explicitamente no estilo?

Gabarito Comentado: Diferente das imagens locais carregadas via `require()` (onde o empacotador Metro inspeciona o arquivo e calcula suas dimensões em tempo de build), imagens de rede não têm suas dimensões conhecidas previamente. Sem largura e altura explícitas, o layout assume dimensões 0x0 e a imagem não é exibida na tela.

Questão 3: Diferencie `<ScrollView>` de `<FlatList>` sob a ótica de renderização e consumo de memória do dispositivo móvel.

Gabarito Comentado: O `<ScrollView>` instancia e desenha todos os itens da lista simultaneamente na memória RAM, independentemente de estarem ou não visíveis no display, causando lentidão e travamentos em listas grandes. Já a `<FlatList>` utiliza **virtualização**, mantendo em memória e desenhando na tela apenas os itens visíveis no viewport (e uma pequena margem de buffer), reciclando componentes durante o scroll e garantindo alta performance mesmo com milhares de registros.

Questão 4: Um aluno escreveu o seguinte código para incrementar um contador:

```
const [contador, setContador] = useState(0);
function aoClicar() {
  contador = contador + 1;
}
```

Por que a tela não atualiza o número exibido quando o usuário clica no botão?

Gabarito Comentado: Porque o estado do React é imutável e a variável `contador` é tratada como somente leitura. Alterar a variável diretamente não notifica o motor do React. Para disparar a re-renderização do componente e atualizar a interface gráfica, deve-se obrigatoriamente chamar a função atualizadora: `setContador(contador + 1)`.

Questão 5: Para carregar dados de uma API externa (via `fetch`) assim que o usuário abre a tela, como deve ser configurado o hook `useEffect`?

Gabarito Comentado: O hook deve ser configurado com um **array de dependências vazio (`[]`)** no segundo argumento: `useEffect(() => { fetchDados(); }, [])`. Isso garante que a função executará exatamente uma vez após a montagem inicial do componente, prevenindo loops infinitos de requisições.

Questão 6: Quais são os 3 navegadores fundamentais do React Navigation e quando cada um deve ser adotado?

Gabarito Comentado: 1. **Stack Navigator:** Navegação empilhada linear (LIFO), com botão de voltar automático. Ideal para fluxos como Lista → Detalhe → Checkout.
2. **Bottom Tab Navigator:** Abas fixas na parte inferior. Ideal para alternar rapidamente entre as áreas principais do app.
3. **Drawer Navigator:** Menu lateral deslizante (gaveta). Ideal para opções secundárias, menus de perfil, configurações ou saídas.

11. Checklist Rápido de 5 Minutos (Decoreba Estratégica)

1. SEM HTML Nada de `<div>`, `<p>` ou `<span>`. Use `<View>` e `<Text>`.

2. TEXTO Todo texto OBRIGATORIAMENTE deve estar encapsulado dentro de `<Text>`.

3. FLEXBOX Eixo padrão é SEMPRE `flexDirection: 'column'`. `justifyContent` = eixo principal; `alignItems` = transversal.

4. COMUNICAÇÃO Dados descem (Pai → Filho por props); Eventos sobem (Filho → Pai por callbacks).

5. PROP DRILLING Repasse excessivo de props por componentes do meio que não as usam. Resolvido por Context / Redux.

6. FLATLIST As 3 props obrigatórias: `data`, `renderItem` e `keyExtractor`.

7. SOMBRAS iOS usa `shadowColor/Offset/Opacity/Radius`. Android usa apenas `elevation`.

8. ESTADO Nunca faça `contador++`. Sempre use `setContador(...)`.

9. USEEFFECT Sem array = roda sempre (loop). Array vazio `[]` = roda 1x na montagem. Return = função cleanup.

10. NAVEGAÇÃO Raiz sempre envolve `<NavigationContainer>`. Stack (pilha), Tabs (abas), Drawer (gaveta).

11. VÍDEO `expo-video` é a biblioteca atual (substituiu o `expo-av` descontinuado). `useVideoPlayer` comanda; `<VideoView>` exibe.